

Introduction to C: Module 2

Weekly Reading: [Beej Guide](#), Chapter 3-4

Anatomy of a C Program

Let's look at a complete C program. You're encouraged to copy it, compile it locally, and verify that it works.

```
#include <stdio.h>

void put_int(int x) {
    printf("%d\n", x);
}

void say_hello(char* name, int age) {
    puts("Hello");
    puts(name);
    puts("You were born in ");
    put_int(2025 - age);
}

int main() {
    say_hello("Bob", 31);
    return 0;
}
```

When compiled, it generates an executable that can be run to print:

```
Hello
Bob
You were born in
1994
```

to the console. The output ain't pretty (we'll get back to that) but we see that the program does something useful—knowing Bob's age, it estimates his year of birth.

The `#include` directive at the top of the file allows us to refer to functions from `stdio.h`: `puts` and `printf`; without it, you wouldn't be able to use them.

In C, a program's entry point is its `main` function. Our `main` is simple: It calls a helper function called `say_hello` and then returns 0, indicating normal exit. The program's actual work occurs in these helper functions, and the type signature of `say_hello` might seem strange. An `int` is an integer (whole number) but what is `char*`? It turns out that C has no native string type.

Instead, we pass the address of (or a *pointer* to) a character array. When the compiler sees the string literal "Bob", it does two things:

1. Includes in the program's memory image a character array (or `char[ ]`) that holds `{ 'B', 'o', 'b', '\0' }`; the final character is a *null terminator* to indicate that the string has ended.
2. At runtime, resolves the literal to a pointer to the memory where the array is stored.

Not all character pointers are valid strings, but all strings are referred-to using the address (pointer) of their initial character. If this doesn't make sense yet, don't worry: We'll cover pointers later.

In this case, our helper function `say_hello` makes calls to the standard library's `puts` (short for "put string") and a `put_int` function that is also included in the code.

How Functions Work

Functions are statically and explicitly typed. A function declaration of:

```
double square_root(int x) {  
    // ...  
}
```

means that `square_root` takes an `int` (integer) and returns a `double`—a 64-bit floating-point number. This is backwards relative to modern notation; in a language like OCaml or Scala, we'd write it as "`int -> double`"; but in C, the return type is written first.

A function that doesn't return a value—it is used only for effects—is given the return type `void`. The function `say_hello`, above, writes to the console, but we don't need any information, so we have it "return `void`."

Let's look at a more interactive, useful program. Again, you're encouraged to copy, compile, run, and tinker with it.

```
// square_demo.c  
  
#include <stdio.h>  
#include <stdlib.h>  
  
int square(int x) {  
    return x * x;  
}  
  
int main(int argc, char** argv) {
```

```

int default_val = 137;
int input = default_val;

if (argc > 1) {
    input = atoi(argv[1]);
}

int result = square(input);
printf("%d squared is %d\n", input, result);
return 0;
}

```

We can compile it...

```
$ gcc -o square_demo square_demo.c
```

and run it:

```
$ ./square_demo 15
15 squared is 225
```

```
$ ./square_demo
137 squared is 18769
```

This is more complicated than the previous example, with a `main` that accepts command-line arguments. The integer `argc` will always be given the total number of arguments, while the `char**` called `argv` (think of it as a string array) allows the program to access them.

Arrays are zero-indexed—if `argc` is 2, you can legally access `argv[1]`, but not `argv[2]`—and `argv[0]` always refers to the name of the program itself—in this case, `./square_demo`. This program, when `argc` is 1, uses a default value of 137 to square; when `argc` exceeds 1, it uses `atoi` to convert `argv[1]` into an integer.

All Base Types Are Numbers

The four most important base types in C are `char`, `int`, `double`, and `size_t`.

A `char` is a one-byte integer type; it's misnamed because:

- characters these days [can be longer than one byte](#)—it's no longer adequate for text.
- it has use cases other than text.

It can safely handle all numbers in the range `[0..127]`, which is enough range to deal with ASCII characters. You might notice that that's only a 7-bit range—that's because it's unspecified

whether the default char type is signed (range [-128..127]) or unsigned ([0..255]). Consider the program below:

```
#include <stdio.h>

int main () {
    char a = 0x80;    // 128 if char unsigned, -128 if char signed
    char b = 0x7f;    // 127
    if (a < b) {
        printf("char is signed\n");
    } else {
        printf("char is unsigned\n");
    }
    return 0;
}
```

This program's behavior is platform-dependent. It assigns the bit pattern 10000000 to a, and 01111111 to b—that, you can rely on—but leaves it to implementation how the former bit pattern is interpreted. It will be -128 (< 127) if characters are signed and 128 if they are unsigned.

Thus, a program that relies on char being signed or unsigned is not portable—always specify, if you depend on a larger range than [0..127], which kind you are using, as is done below:

```
#include <stdio.h>

int main () {
    unsigned char a = 0x80;    // 128 (unambiguous)
    unsigned char b = 0x7f;    // 127
    if (a < b) {
        assert(0);            // never get here
    } else {
        printf("OK\n");
    }
    return 0;
}
```

Because char is an integer type, it is legal to write:

```
char x = 65;    // prefer when you're working with tiny numbers
```

or:

```
char x = 'A';    // prefer when you're working with text
```

or:

```
unsigned char x = 0x41; // prefer when working w/ binary data
```

They are all equivalent—'A' is just an alias for 65 (hexadecimal 0x41).

Another type you will see often used is `int`, which is inescapable. It's a signed integer of... unspecified size. In other words, this detail is *also* implementation dependent. It is guaranteed to handle all values between -32,767 and +32,767—this requires 2 bytes—but most systems these days use 4 bytes, giving a much larger range. Historically, `int` was the same size as the machine word, but modern computers have 64-bit (8-byte) words and still use 4-byte `ints` for reasons of historical compatibility. You can think of `int` as *usually* 32 bits long—but it's not guaranteed.

C has absolutely no overflow protection. For a contrast, Python automatically checks integer operations for overflow and promotes them to larger types when out-of-bounds results are produced:

```
$ python
```

```
...
>>> 65537 ** 5                # result will be too large for a 64-bit int
1209018056149790439571457
```

C doesn't do this for you. It's expensive, and in high-performance code, you often don't want overflow checks on every single math operation. If you need it in C, you have to write it yourself.

Unsigned integers have “odometer” behavior in the case of overflow—`UINT_MAX + 1 == 0`—whereas signed integers have *undefined behavior*, which we'll discuss in detail in the next module. Just know that you cannot make any assumptions about program semantics when undefined behaviors exist—so they are *always* bugs.

Fixed-size integer types are available if you `#include <inttypes.h>`. For example, `uint16_t` will always be an unsigned 16-bit integer, and `int32_t` will always be a signed 32-bit one. If your program depends on specific signedness or sizes, always use those. When you use `int`, you're letting the system and compiler decide.

When you want floating-point (fractional) numbers, use `float` and `double`. These correspond to 32- and 64-bit floating-point numbers, respectively. Your default tool will usually be the double-precision option—the smaller floats are used in graphics and machine learning, but they won't be covered here.

Finally, `size_t` is an unsigned integer that exists to contain the size (in bytes) of an object such as an array or file. Of course, such objects can be huge, so `size_t` is usually 8 bytes, or 64 bits, capable of holding any number from 0 to 18,446,744,073,709,551,615 ($2^{64} - 1$).

You can learn what these sizes are on your system by using the `sizeof` operator, which determines at compile time the size (in bytes) of a type or value.

```
#include <stdio.h>

int main() {
    printf("Size of char: %lu\n", sizeof(char));
    printf("Size of int: %lu\n", sizeof(int));
    printf("Size of size_t: %lu\n", sizeof(size_t));
    return 0;
}
```

Your results, on a modern desktop computer, will very likely be identical to mine:

```
$ ./m2_sizes
Size of char: 1
Size of int: 4
Size of size_t: 8
```

Control Flow

Conditional execution is similar to that in other languages. This Python code, for example:

```
if x < 100:
    a()
elif x < 200:
    b()
else:
    c()
```

would be written in C like so:

```
if (x < 100) {
    a();
} else if (x < 200) {
    b();
} else {
    c();
}
```

There is no `elif`, parentheses around branching conditions are mandatory, and blocks are separated by curly braces rather than indentation, but they are conceptually the same.

Conditional loops, likewise, are similar to those in Python, so:

```
while x < 0:
    f()
    g()
```

has the C counterpart of:

```
while (x < 0) {
    f();
    g();
}
```

Historically, C used integer types in lieu of booleans—you will often see `while (1)` as a counterpoint to Python's `while True`—with 0 representing falsity and all nonzero values indicating truth. These days, however, you should use the `bool` type and the constants `true` and `false` that become available if you `#include "stdbool.h"`.

C does not have the list comprehension semantics of Python's `for`; the behavior of the `for`-loop, which will be familiar to Java users, is like so:

```
for (int i = 0; i < 100; i++) {           // for (initialization; test; increment)
    f(i);                                //      loop body
}
```

is equivalent to:

```
int i = 0;
while (i < 100) {
    f(i);
    i++;
}
```

where `i++` ("increment operator") is shorthand `i += 1` or `i = i + 1`.

C has `break` and `continue` with identical semantics to their counterparts in Python—`break` exits the nearest enclosing loop immediately, while `continue` returns to the top of it. Thus, the following loops:

```
while (a < 100) {
    f();
}
```

and

```
while (true) {
    if (a >= 100) break;
    f();
}
```

are equivalent. From a stylistic perspective, prefer the former, as it's more idiomatic and therefore easier to understand. There are some who avoid `break` and `continue`; I will say that they are occasionally useful, but should be used with caution.

C has an idiom you might not have seen in other languages—the `do-while` loop, which places the test at the end of the block, ensuring that the loop runs at least once. Thus:

```
do {
    f();
} while (a < 100);
```

is equivalent to:

```
while (true) {
    f();
    if (a >= 100) break;
}
```

or

```
f();
while (a < 100) {
    f();
}
```

Note that the `do-while` construction requires a semicolon at the end, whereas ordinary `while-loops` do not.

The `switch` statement will be familiar to Java users, but new to Python programmers. It evaluates an integer expression and dispatches on the value. For example:

```
switch (suit) {
    case 0:
        printf("Diamond\n");
        break;
    case 1:
        printf("Spade\n");
```



```

        break;
    case 2:
        printf("Heart\n");
        break;
    case 3:
        printf("Club\n");
        break;
    default:
        printf("(no suit)\n")
}

```

If suit is 2, then this block will print "Heart" to the console. **The break statement is necessary**; if it weren't there, then execution would "fall through" to execute the case 3 code—in this way, switch blocks can be thought of as a limited version of the hated "goto" statement. Control flow jumps to the case 2 label, and exits via break to avoid doing more work. The default case is a catch-all for values that don't match any case—if it isn't provided, then nothing happens in the no-match case.

To do this in Python, you would use a cascading if-else tree:

```

if suit == 0:
    print('Diamond')
elif suit == 1:
    print('Spade')
elif suit == 2:
    print('Heart')
elif suit == 3:
    print('Club')
else:
    print('(no suit)')

```

To be honest, there's nothing wrong with this. The advantage of a switch/case block is performance. The if/else cascade requires a check for each value, and is $O(n)$ in the number of cases. However, a compiler can, when it is faster to do so, replace the serial checks with an $O(1)$ table-based approach.

Week 2 Questions

2.1: What do you think the following loop does? Make your guess, then run a program to test your intuition.

```
unsigned int i = 1;
while (i > 0) {
    i++;
}
```

2.2: Would you expect the behavior of the loop above to change if a regular (signed) `int` were used? Can you think of any circumstances in which it might not do that?

2.3: Investigate `sizeof`. Is this a function, or something else? If it's not a function, then how is it different?

Week 2 Project

Create a command line utility that tabulates the letters in all words (which do not have to be valid English words) supplied as arguments and prints out the number of each letter in the whole list. All words will be of length 1–128, in lower-case ASCII letters, and there will be no more than 255 of them.

Example usages:

```
$ ./tabulate
```

```
$ ./tabulate bat cat dog
```

```
A 2
```

```
B 1
```

```
C 1
```

```
D 1
```

```
G 1
```

```
O 1
```

```
T 2
```

```
$ ./tabulate aaaa bxvr corn dzqwrqnx
```

```
A 4
```

```
B 1
```

```
C 1
```

```
D 1
```

```
N 2
```

```
O 1
```

```
Q 2
```

R 3
V 1
W 1
X 2
Z 1

2.4. Include your code for this exercise.

Submit your answers in PDF form by **September 4**.